绘制文字

绘制文字
- 例程简介
- API 文档
  - `draw_string_advanced`
  - `draw_string`
- 示例代码
- 例程运行效果

例程简介

本节我们介绍绘制文字的draw_string_advanced方法

API 文档

【推荐使用】

`draw_string_advanced`

增强版 `draw_string`，支持中文显示，并允许用户通过 `font` 参数自定义字体。

```
image.draw_string_advanced(x, y, char_size, str, [color, font])
```

【不推荐使用】

`draw_string`

```
image.draw_string(x, y, text[, color[, scale=1[, x_spacing=0[, y_spacing=0[,
mono_space=True]]]]])
```

从图像的 `(x, y)` 位置开始绘制 8x10 大小的文本。参数可以分别传入 `x, y`，也可以作为元组 `(x, y)` 一起传递。

- **text**: 要绘制的字符串，换行符 `\n`、`\r` 或 `\r\n` 用于将光标移动到下一行。
- **color**: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- **scale**: 控制文本的缩放比例，默认为 1。只能为整数。
- **x_spacing**: 调整字符之间的水平间距。正值表示增加间距，负值表示减少。
- **y_spacing**: 调整行之间的垂直间距。正值表示增加间距，负值表示减少。
- **mono_space**: 默认为 `True`，使字符具有固定宽度。设置为 `False` 时，字符间距将根据字符宽度动态调整。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

示例代码

```
```

```python
# Import required modules
# 导入所需的模块
import time, os, urandom, sys, math

# Import display and media related modules
# 导入显示和媒体相关模块
from media.display import *
from media.media import *

# Define display resolution constants
# 定义显示分辨率常量
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480

def display_test():
    """
    Function to test display functionality
    测试显示功能的函数
    """

    # Create main background image with white color
    # 创建白色背景的主图像
    img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)
    img.clear()
    img.draw_rectangle(0, 0, DISPLAY_WIDTH, DISPLAY_HEIGHT,color=
(255,255,255),fill=True)

    # Initialize display with ST7701 driver
    # 使用ST7701驱动初始化显示器
    Display.init(Display.ST7701, width = DISPLAY_WIDTH, height = DISPLAY_HEIGHT,
to_ide = True)
    # Initialize media manager
    # 初始化媒体管理器
    MediaManager.init()

    try:
        # 中文 - 使用翠绿色
        img.draw_string_advanced(300, 240, 30, "你好，世界！", color=(0, 255, 127))
        # 英语 - 使用天蓝色
        img.draw_string_advanced(300, 180, 30, "Hello World!", color=(0, 191,
255))
        # Update display with background image
        # 更新显示背景图像
        Display.show_image(img)
        while True:
            time.sleep(2)

    except KeyboardInterrupt as e:
        print("user stop: ", e)
    except BaseException as e:
        print(f"Exception {e}")

    # Cleanup and deinitialize display
    # 清理并反初始化显示器
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
    # Release media resources
```

```
    # 释放媒体资源
    MediaManager.deinit()

if __name__ == "__main__":
    # Enable exit points and run display test
    # 启用退出点并运行显示测试
    os.exitpoint(os.EXITPOINT_ENABLE)
    display_test()
```

例程运行效果

可以看到，我们在屏幕的中心用直线绘制了一个“Hello World”和“你好，世界！”的字符串

